

Índice:

- line that just executed
- next line to execute

- Linha que acaba de ser executada
- Próxima linha a ser executada

Representação da memória com seus endereços:

0	4	8	C	10	14	18	1C	20	24	28	2C	30	34	38
3C	40	44	48	4C	50	54	58	5C	60	64	68	6C	70	74
78	7C	80	84	88	8C	90	94	98	9C	A0	A4	A8	AC	B0
B4	B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC

Estados da memória durante a execução do programa:

Execução do programa:

```
➡ 17 int main() {
18   int valor = 12345;
19
20   printf("Invertendo o numero %d: ", valor);
21   inverterNumero(valor);
22   printf("\n");
23
24   return 0;
25 }
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
98	9C	A0	A4	A8
D4	D8	DC	E0	E4
valor				
?				

. O programa iniciará a partir do “int main()”, função principal.

Execução do programa:

```
➡ 17 int main() {
➡ 18   int valor = 12345;
19
20   printf("Invertendo o numero %d: ", valor);
21   inverterNumero(valor);
22   printf("\n");
23
24   return 0;
25 }
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
98	9C	A0	A4	A8
D4	D8	DC	E0	E4
		valor		
		?		

Quando o programa inicia sua execução a memória reserva um endereço para a variável “valor”, mas ainda não há valor armazenado.

Execução do programa:

```
17 int main() {
➡ 18   int valor = 12345;
19
➡ 20   printf("Invertendo o numero %d: ", valor);
21   inverterNumero(valor);
22   printf("\n");
23
24   return 0;
25 }
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
98	9C	A0	A4	A8
D4	D8	DC	E0	E4
		valor		
		12345		

O valor “12345” então é armazenado na variável que passa a ter endereço definido na memória.

Execução do programa:

```
→ 20 printf("Invertendo o numero %d: ", valor);
→ 21 inverterNumero(valor);
22 printf("\n");
23
24 return 0;
25 }
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
98	9C	A0	A4	A8
D4	D8	DC	E0	E4
		valor		
		12345		

Foi exibida uma mensagem e a memória manteve seu estado.

Na próxima linha a função “inverterNumero(valor)” será chamada com a variável valor sendo “passada” como parâmetro.

Execução do programa:

```
→ 4 void inverterNumero(int n) {
5 // Caso base: se o número for 0, não faz nada e começa a voltar
6 if (n == 0) {
7 return;
8 }
9
10 // Ação: imprime o último dígito (resto da divisão por 10)
11 printf("%d", n % 10);
12
13 // Chamada recursiva: passa o número sem o último dígito
14 inverterNumero(n / 10);
15 }
16
17 int main() {
18 int valor = 12345;
19
20 printf("Invertendo o numero %d: ", valor);
→ 21 inverterNumero(valor);
22 printf("\n");
}
```

Estado da memória:

0	4	8	C	10
3C	40	44	48	4C
78	7C	80	84	88
B4	B8	BC	C0	C4
		n		
		?		

A função é chamada e o seu conteúdo será executado. Em seu conteúdo há uma variável (n) “visível” apenas para a própria função e a memória prontamente faz a alocação do espaço necessário.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
→ 4 void inverterNumero(int n) {
5 // Caso base: se o número for 0, não faz nada e começa a voltar
→ 6 if (n == 0) {
7 return;
8 }
9
10 // Ação: imprime o último dígito (resto da divisão por 10)
11 printf("%d", n % 10);
12
13 // Chamada recursiva: passa o número sem o último dígito
14 inverterNumero(n / 10);
15 }
16
```

Estado da memória:

0	4	8	C	10
3C	40	44	48	4C
78	7C	80	84	88
B4	B8	BC	C0	C4
		n		
		12345		

O valor “12345” é armazenado na variável “n”, que já possui seu endereço definido.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
```

Estado da memória:

0	4	8	C	10
3C	40	44	48	4C
78	7C	80	84	88
B4	B8	BC	C0	C4
		n		
		12345		

É realizada uma comparação no if, caso o número seja 0 retorna a partir desse ponto. Como o número é maior que 0, o programa segue o fluxo da função, imprime o último dígito através do resto da divisão por 10 e a memória mantém o seu estado.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
16
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
98	9C	A0	A4	A8
	n			
	?			
D4	D8	DC	E0	E4
		valor		
		12345		

Através de uma operação aritmética (divisão por 10), o último dígito é retirado, já que pelo fato de estar armazenado como número inteiro, tudo o que vem depois da vírgula é ignorado pelo programa, então o dígito “some”.

A memória já reserva um endereço para armazenar o resultado dessa operação e utiliza-lo dentro da função.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
98	9C	A0	A4	A8
	n			
	1234			
D4	D8	DC	E0	E4
		valor		
		12345		

O valor “1234” é armazenado na variável “n”, no endereço de final: “9C”.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5 // Caso base: se o número for 0, não faz nada e começa a voltar
6 if (n == 0) {
7 return;
8 }
9
10 // Ação: imprime o último dígito (resto da divisão por 10)
11 printf("%d", n % 10);
12
13 // Chamada recursiva: passa o número sem o último dígito
14 inverterNumero(n / 10);
15 }
16
```

Estado da memória:

0	4	8	C	10
3C	40	44	48	4C
78	7C	80	84	88
	n			
	?			
B4	B8	BC	C0	C4
		n		
		12345		

O fluxo da função é retomado novamente, a comparação é realizada no if, o programa segue o fluxo da função (por ser maior que zero), o último dígito é “printado” através do resto da divisão por 10 e é dividido por 10 para a retirada de mais um dígito.

A memória aloca o endereço de final 7C para o armazenamento da variável e seu respectivo valor, ainda sem definição.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5 // Caso base: se o número for 0, não faz nada e começa a voltar
6 if (n == 0) {
7 return;
8 }
9
10 // Ação: imprime o último dígito (resto da divisão por 10)
11 printf("%d", n % 10);
12
13 // Chamada recursiva: passa o número sem o último dígito
14 inverterNumero(n / 10);
15 }
16
```

Estado da memória:

0	4	8	C	10
3C	40	44	48	4C
78	7C	80	84	88
	n			
	123			
B4	B8	BC	C0	C4
		n		
		12345		

Novamente, dentro da função uma comparação é realizada no if, o programa segue o fluxo da função (por ser maior que zero), o último dígito é “printado” através do resto da divisão por 10 e é dividido por 10 para a retirada de mais um dígito.

A memória insere o valor “123” dentro da variável no endereço de final 7C.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
16
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
<i>n</i>				
?				
98	9C	A0	A4	A8
	<i>n</i>			
	1234			
D4	D8	DC	E0	E4
		<i>valor</i>		
		12345		

Ao iniciar o fluxo da função novamente uma comparação é realizada no if, o programa segue o fluxo da função (por ser maior que zero), o último dígito é “printado” através do resto da divisão por 10 e é dividido por 10 para a retirada de mais um dígito.

A memória aloca o endereço de final 5C para o armazenamento da variável e seu respectivo valor, ainda sem definição.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
16
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
<i>n</i>				
12				
98	9C	A0	A4	A8
	<i>n</i>			
	1234			
D4	D8	DC	E0	E4
		<i>valor</i>		
		12345		

Novamente, dentro da função uma comparação é realizada no if, o programa segue o fluxo da função (por ser maior que zero), o último dígito é “printado” através do resto da divisão por 10 e é dividido por 10 para a retirada de mais um dígito.

A memória insere o valor “12” dentro da variável no endereço de final 5C.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
16
```

Estado da memória:

0	4	8	C	10
3C	40	44	48	4C
n				
?				
78	7C	80	84	88
	n			
	123			
B4	B8	BC	C0	C4
		n		
		12345		

Ao iniciar o fluxo da função novamente uma comparação é realizada no if, o programa segue o fluxo da função (por ser maior que zero), o último dígito é “printado” através do resto da divisão por 10 e é dividido por 10 para a retirada de mais um dígito.

A memória aloca o endereço de final 3C para o armazenamento da variável e seu respectivo valor, ainda sem definição.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
16
```

Estado da memória:

0	4	8	C	10
3C	40	44	48	4C
n				
1				
78	7C	80	84	88
	n			
	123			
B4	B8	BC	C0	C4
		n		
		12345		

Novamente, dentro da função uma comparação é realizada no if, o programa segue o fluxo da função (por ser maior que zero), o último dígito é “printado” através do resto da divisão por 10 e é dividido por 10 para a retirada de mais um dígito.

A memória insere o valor “1” dentro da variável no endereço de final 3C.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
16
```

Estado da memória:

18	1C	20	24	28
	<i>n</i>			
	?			
54	58	5C	60	64
		<i>n</i>		
		12		
90	94	98	9C	A0
			<i>n</i>	
			1234	
CC	D0	D4	D8	DC
				<i>valor</i>
				12345

Ao iniciar o fluxo da função novamente uma comparação é realizada no if, o programa segue o fluxo da função (por ser maior que zero), o último dígito é “printado” através do resto da divisão por 10 e é dividido por 10 para a retirada de mais um dígito.

A memória aloca o endereço de final 1C para o armazenamento da variável e seu respectivo valor, ainda sem definição.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5     // Caso base: se o número for 0, não faz nada e começa a voltar
6     if (n == 0) {
7         return;
8     }
9
10    // Ação: imprime o último dígito (resto da divisão por 10)
11    printf("%d", n % 10);
12
13    // Chamada recursiva: passa o número sem o último dígito
14    inverterNumero(n / 10);
15 }
16
```

Estado da memória:

18	1C	20	24	28
	<i>n</i>			
	0			
54	58	5C	60	64
		<i>n</i>		
		12		
90	94	98	9C	A0
			<i>n</i>	
			1234	
CC	D0	D4	D8	DC
				<i>valor</i>
				12345

Novamente, dentro da função uma comparação é realizada no if, porém agora a condição (n==0) é verdadeira, levando o programa ao comando “return” o que encerra a execução da função.

Execução do programa:

```
1 #include <stdio.h>
2
3 // Função que imprime os dígitos de um número na ordem inversa
4 void inverterNumero(int n) {
5 // Caso base: se o número for 0, não faz nada e começa a voltar
6 if (n == 0) {
7 return;
8 }
9
10 // Ação: imprime o último dígito (resto da divisão por 10)
11 printf("%d", n % 10);
12
13 // Chamada recursiva: passa o número sem o último dígito
14 inverterNumero(n / 10);
15 }
16
17 int main() {
18 int valor = 12345;
19
20 printf("Invertendo o numero %d: ", valor);
```

Estado da memória:

18	1C	20	24	28
	n			
	0			
54	58	5C	60	64
		n		
		12		
90	94	98	9C	A0
			n	
			1234	
CC	D0	D4	D8	DC
				valor
				12345

Agora no “return” o programa realiza a liberação um a um dos endereços de memória antes ocupados pelas variáveis armazenando valores, essa liberação ocorre do ultimo endereço ocupado para o primeiro.

Logo se a ordem do programa ocupou os endereços na seguinte ordem: DC, BC, 9C, 7C, 5C, 3C, 1C.

No momento da “limpeza” de memória a ordem será: 1C, 3C, 5C, 7C, 9C, BC.

Este comportamento da “desalocação” tem uma lógica de pilha de memória

Execução do programa:

```
→ 24 return 0;
→ 25 }
```

Estado da memória:

20	24	28	2C	30
5C	60	64	68	6C
98	9C	A0	A4	A8
D4	D8	DC	E0	E4
		valor		
		12345		

Após a execução da função, ao chegar ao “return 0;” o único endereço de memória ocupado é DC.

O programa encerra.